Received: 27 June 2024 | Accepted: 23 July 2024

DOI: 10.1002/aaai.12198

SPECIAL TOPIC ARTICLE

Framework to enable and test conversational assistant for APIs and RPAs

Jayachandu Bandlamudi[1] | **Kushal Mukherjee**[2] | **Prerna Agarwal**[2] | **Ritwik Chaudhuri**[1] | **Rakesh Pimplikar**[3] | **Sampath Dechu**[4] | **Alex Straley**[4] | **Anbumunee Ponniah**[3] | **Renuka Sindhgatta**[1]

[1]IBM Research—India, Bangalore, Karnataka, India

[2]IBM Research—India, New Delhi, India

[3]IBM India Private Limited, Bangalore, Karnataka, India

[4]IBM, Armonk, New York, USA

Correspondence
Jayachandu Bandlamudi, IBM Research—India, Bangalore, Karnataka, India.
Email: jay_bandlamudi@in.ibm.com

Abstract

In the realm of business automation, conversational assistants are emerging as the primary method for making automation software accessible to users in various business sectors. Access to automation primarily occurs through application programming interface (APIs) and robotic process automation (RPAs). To effectively convert APIs and RPAs into chatbots on a larger scale, it is crucial to establish an automated process for generating data and training models that can recognize user intentions, identify questions for conversational slot filling, and provide recommendations for subsequent actions. In this paper, we present a technique for enhancing and generating natural language conversational artifacts from API specifications using large language models (LLMs). The goal is to utilize LLMs in the "build" phase to assist humans in creating skills for digital assistants. As a result, the system does not need to rely on LLMs during conversations with business users, leading to efficient deployment. Along with enabling digital assistants, our system employs LLMs as proxies to simulate human interaction and automatically evaluate the digital assistant's performance. Experimental results highlight the effectiveness of our proposed approach. Our system is deployed in the IBM Watson Orchestrate product for general availability.

INTRODUCTION AND BACKGROUND

AI-driven automation is leading to the digital transformation of businesses in the fourth industrial revolution era (Laurent, Chollet, and Herzberg 2015). Automation systems with interactive AI capabilities help users to automate repetitive tasks and improve the performance of their business (Devarajan 2018). Application programming interface (API) services and to a greater extent robotic process automation (RPAs) are the mainly adopted automation technologies for automation (Syed et al. 2020).

However, despite their omnipresence, there is an increasing demand to bring APIs and RPAs into conversational automation systems. Task-oriented conversational automation, such as chatbots[1] and digital assistants, represents the next frontier in user engagement and customer support expanding from basic QnA-type tasks to include actions (Yaghoub-Zadeh-Fard, Benatallah, and

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.
© 2024 The Author(s). *AI Magazine* published by John Wiley & Sons Ltd on behalf of Association for the Advancement of Artificial Intelligence.

Zamanirad 2020). Integrating APIs into these systems opens up a world of possibilities, as it allows them to access real-time data, perform complex tasks, and connect with external services on demand. Given that many typical business users do not have the technical know-how in the area of software development and automation, Task-oriented conversational systems alleviate the main difficulty in the adoption of automation technologies.

One way to enhance the accessibility of automation technologies like RPA, especially unattended bots, is by encapsulating their API services within chatbots (Vaziri et al. 2017). Employing natural language for interactive automation can offer a more intuitive experience for business users, as conversational systems strive to communicate in a human-like manner. Traditionally, creating chatbots involves multiple steps including (Bocklisch et al. 2017; Sabharwal et al. 2020) (i) training an intent classifier, which necessitates expertise in machine learning, natural language processing, and domain-specific data from experts; (ii) identification of the inputs and their data types (often called slots) required for invoking the API; and (iii) processing of the result of the API and providing next action recommendations.

LLMs with vast parameter sizes, ranging from millions to billions, have demonstrated promising outcomes in various natural language tasks, including intent recognition and language generation (Zhao et al. 2023). These models are trained on trillions of tokens extracted from open-domain text sources. They have been used to enable a conversation for APIs (Patil et al. 2023) in a few cases. However, when it comes to enterprise automation, specialized domain knowledge is often required, which is not inherently present in the data used for training these LLMs (Yang, Uy, and Huang 2020). Consequently, adjustments or additional contextual information are necessary to make these models relevant to specific domains or prompts. Unfortunately, many businesses, especially small and medium enterprises, face challenges in fine-tuning such models due to limited resources or expertise, making the process prohibitively expensive. Further, LLMs suffer from the problem of uncertain outputs and hallucinations with limited control over what is generated (Bang et al. 2023). In addition to that, there is the issue of the high cost of deployment that requires a lot of infrastructure (including GPUs) to scale.

To alleviate the above issues, we have created a human-guided process of authoring and wrapping API services as Digital Assistant/chatbot skills. This process is called the *build* step. The objective is to use LLMs in the *build* phase to assist the human to author the skills for the digital assistant. This is typically a one-time step for each automation. Subsequently, the system does not have to leverage LLMs while conversing with business users. Human-guided authoring of skills is a preferred approach until before LLMs reach that level of maturity. It also provides a level of trust and protection against indemnity from the client.

In this paper, we propose a method that leverages LLMs to generate conversational artifacts that can be used to train intent classifiers with reduced reliance on AI expertise, generate questions for conversational input slot filling, and derive the next actions for skills. After generating required conversational artifacts, we test the conversational assistant in an automated manner by using an LLM as a proxy user to converse with the assistant and another LLM as the judge to evaluate the conversation.

Our approach involves language generation using the meta-data from the OpenAPI[2] specifications of the APIs.[3] Harnessing the generative capabilities of LLMs offline during *build* step helps eliminate the run-time expenses associated with high-end hardware. Through our experiments, we demonstrate that we can achieve improved performance by using LLMs at *build* time for intent classification, question generation for slot-filling, and next-best agent (NBA) prediction, while simultaneously lowering the barriers for non-AI experts by involving human subject matter experts. Further, our approach of using general-purpose instruct-tuned models ensures that the same model can work for multiple tasks and may be deployed efficiently.

The following are the contributions of the paper: enabling a conversational interface for APIs/RPAs by the generation of natural language conversational artifacts in an agent-based framework that significantly reduces *build* effort; enabling human-guided building of conversational interface for APIs/RPAs for improving trust and protecting against indemnity; use of LLMs to generate data for intent classification, question generation for slot filling, and next action recommendations for invoking APIs/RPAs in conversations; using LLMs as proxy users for simulating user interactions with conversational assistants, and judging the simulated conversation.

SYSTEM AND ARCHITECTURE

We aim to address the challenges mentioned above by enhancing accessibility for users to run automation (APIs/RPAs) and intelligent sequencing of automation. To achieve this goal, we employ a conversational interface that is recognizable to automation users on proprietary platforms. These interfaces enable users to employ natural language to execute automation for addressing business-related tasks. In this paper, we introduce a comprehensive framework or tool that allows all current APIs and RPAs to be seamlessly accessed and utilized through conversational interactions.

To wrap an API into a conversation interface, we make use of a framework for constructing digital assistants, as outlined in Rizk et al. (2020). The authors introduced a framework designed to connect RPAs and corporate chatbots, employing a multi-agent orchestrator that facilitates interactive automation via natural language and AI planning. Next, we introduce the fundamental principles utilized within this framework as described in Rizk et al. (2020).

Skills are the atomic functions, there exist five types of skills in the framework: (i) *understand* user's intention, (ii) *request* information from the user to perform the action (conversational slot filling), (iii) *act* on the request, (iv) *respond* to the user, and (v) provide recommendations for next best agent. In detail, the *understand* skill refers to the natural language understanding of a user's utterances. The *request* skill asks natural language questions to the user for gathering the required inputs. The *act* skill, in our context, is invoking the API/RPA with the gathered inputs. The *respond* skill is for providing a human-consumable response based on the output of the *act* skill. Finally, the *NBA* skill provides recommendations for the NBAs that the user may execute. ***Agents*** are a composition of *understand–request–act–respond–NBA* skills that transform the set of skills into a conversational entity. Agents receive a user utterance and the current context of the execution or state. They act on the utterance based on the context and respond with an updated context and confidence in their response, indicating their comprehension of the input and response. ***Orchestrator*** handles interaction between agents, that include *human* agents. The main function of an orchestrator is to (i) select the correct agent from a predefined set of available agents to achieve a task, (ii) manage the context, and (iii) manage the flow of the conversation between the agents. The orchestrator receives a user utterance and forwards it to all agents available along with appropriate context from the user. Once it receives the agents' preview response, it chooses the most appropriate agent (or sequence of agents) according to user utterance. After selecting the agent, it connects with the agent to complete the execution and manage the context.

Thus, the process of wrapping an API operation in a conversational interface involves the authoring of the corresponding agent specification. The agent specification requires a set of natural language conversational artifacts derived from the API metadata such as (i) the *understand* skill requires the generation of multiple utterances based on the description of the API operations. These utterances are then used to train the intent classifier. (ii) The *request* skill requires the generation of contextual questions to request user information to execute the task. These questions have to be generated based on the API metadata including description and input–output schema. (iii) The *NBA* skill to obtain possible next agents from the available set of agents and also perform semantic mapping of inputs and outputs of agents to sequence them.

Our system architecture, described in Figure 1, shows two personas: one is the "builder" and the other is the "user." The "builder" persona selects what APIs or RPAs have to be exposed via the conversational interface, that is, the creation of the agent. Whereas the "user" persona uses the APIs or RPAs via the conversational interface. When a "builder" adds a new API, it is first validated and captured in a catalog and then the *build* phase is triggered. The three components of *build* phase are used to generate the required conversational artifacts of an agent specification. Once an agent is generated, it is stored in the agent database. It is then evaluated by the Automated Agent Testing component, which sends the verdict on agent to "builder." The "builder" can review and update (if required) before publishing the agent. Figure 2 shows how a builder would choose a service and review/update the conversational artifacts. Figure 3 shows a sample agent specification with the conversational artifacts marked in red color.

Once the agents are published, they are available to the "users" of the system and invoked through a user utterance. The orchestrator (Rizk et al. 2020) receives an utterance from the user and invokes the *understand* skill of the agents. The *understand* skill classifies the utterance to one of the agents. In cases where the classifier confidence is low or there are multiple agents with similar confidences, an appropriate fallback or disambiguation action is taken (Rizk et al. 2020). Once the correct agent is selected, the orchestrator executes the sequence of skills *request–act–respond–NBA*. Figure 4 shows the "users" conversational experience. If the orchestrator selects an in-correct agent, then user can cancel the execution of agent. During *use* time, there is no need for expensive inference on LLMs as all the conversational artifacts have been pregenerated, tested, and reviewed in the *build* phase.

UTTERANCE GENERATION

The Orchestrator leverages the *understand* skill of agents to identify the correct agent according to the provided user utterance. The *understand* skill consists of an intent classifier trained in the *build phase* of the corresponding agent as shown in Figure 3. During the *build* phase, we obtain the agent description from the corresponding API operation and it is to be noted that only one description is provided for each agent. Since a single description as the training sample is not sufficient to train the intent classifier, the training data are augmented via utterance generation. As shown in Figure 5, utterance generation involves several

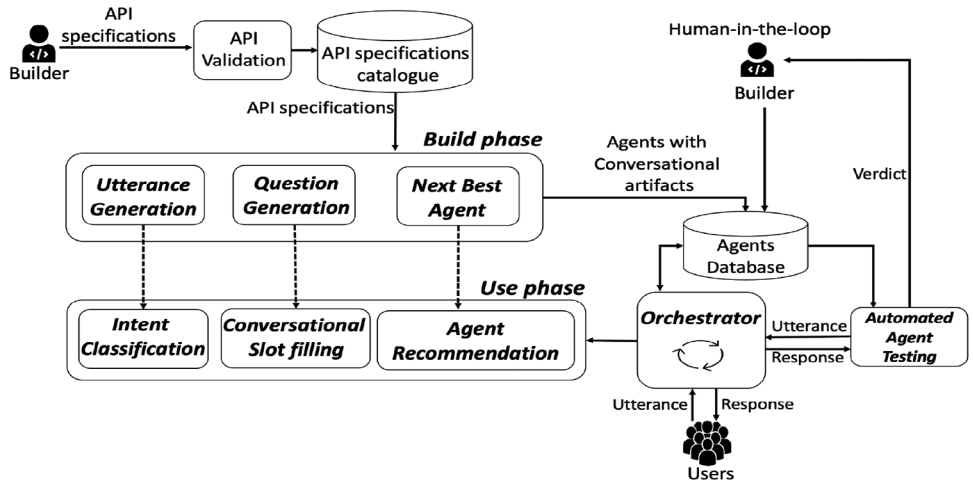


FIGURE 1 System architecture.

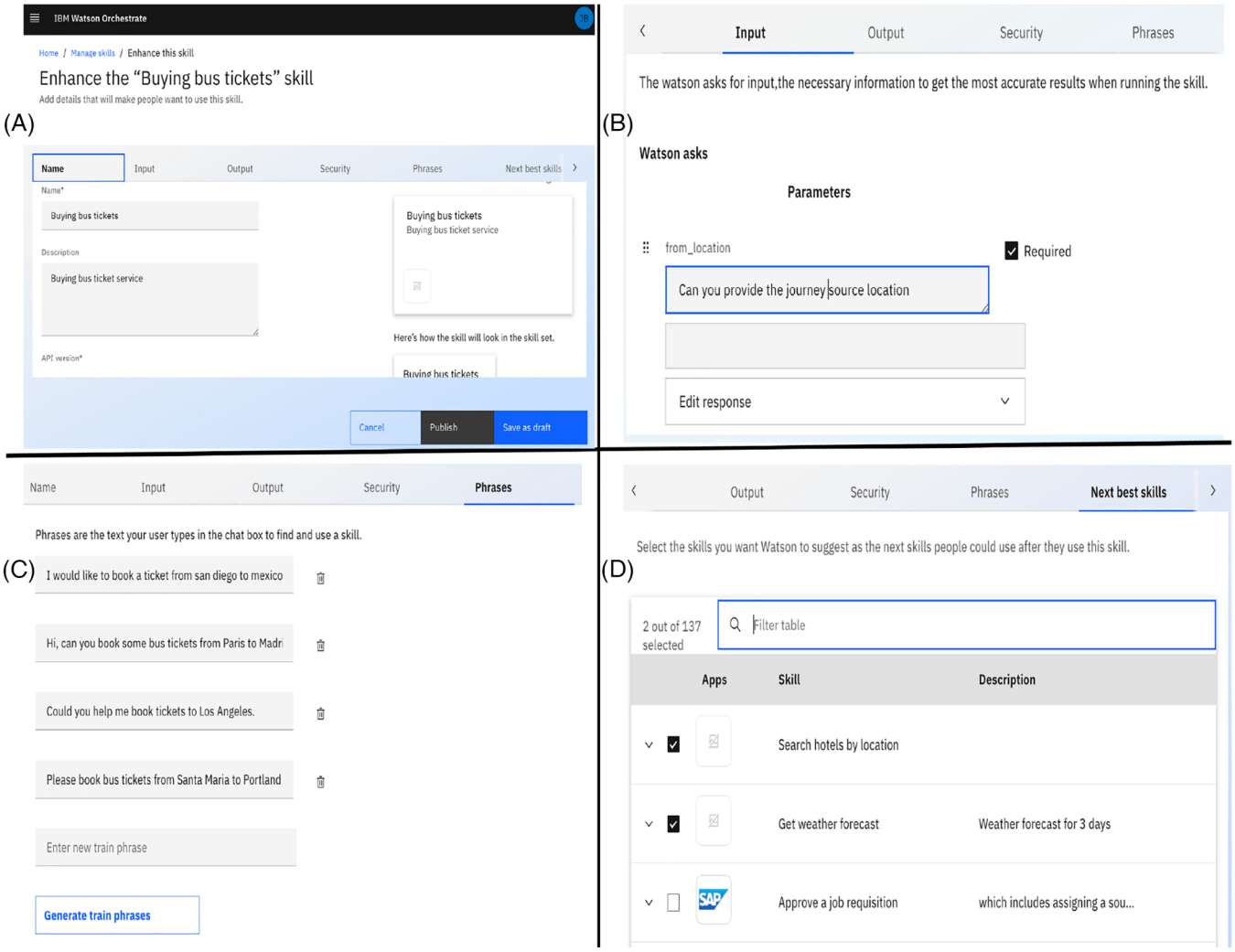


FIGURE 2 System builder interface. (A) The process of adding a new skill from an API specification; (B) LLM generated questions are displayed for each attribute of the API call that the builder reviews/updates; (C) generated utterances that would be used to train the intent classifier; (D) next actions are displayed to be reviewed/updated by the builder.

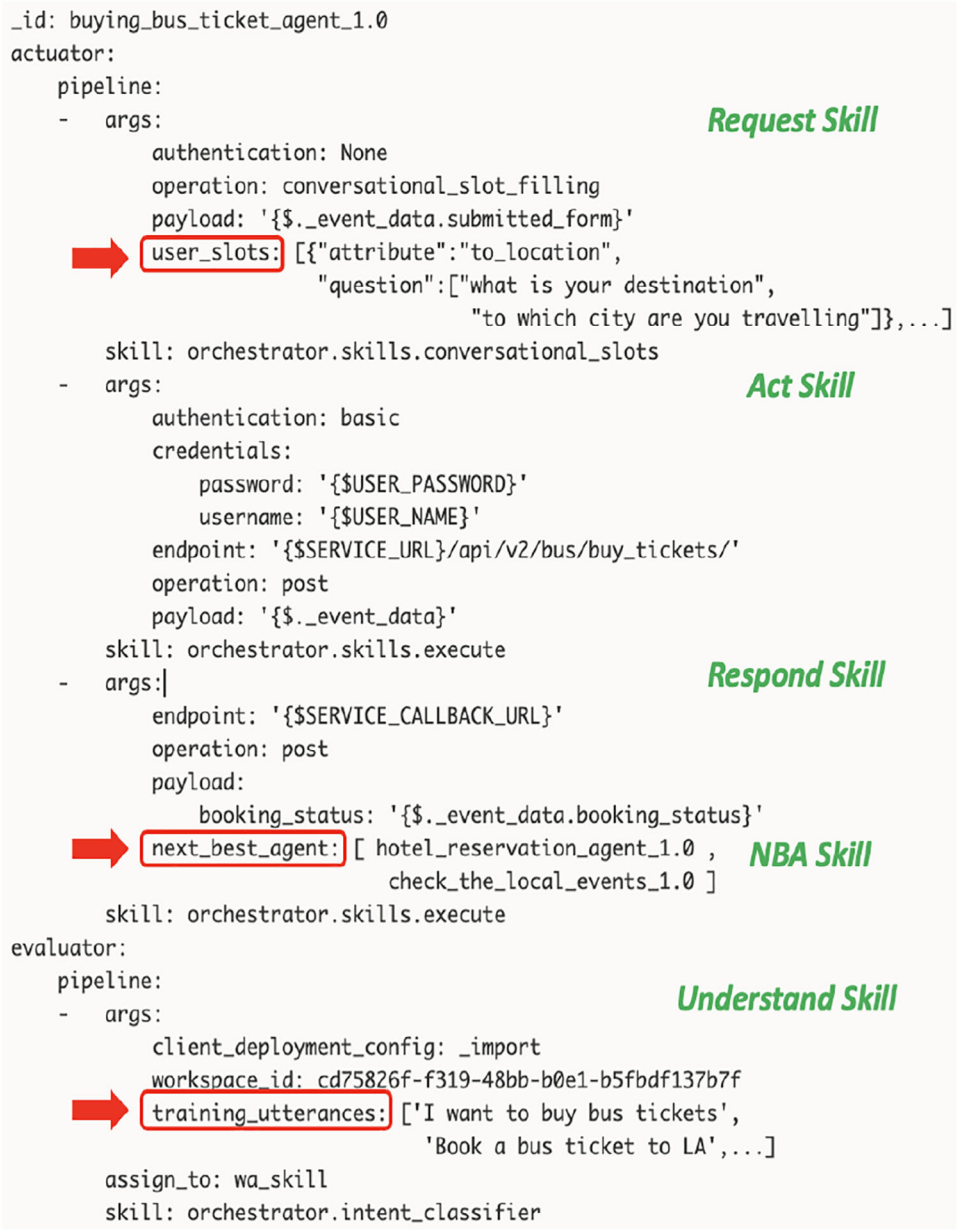


FIGURE 3 Example agent with skills and the generated conversational artifacts marked in red.

steps. At first, we obtain the agent description from the API specification. Using the agent description and an LLM prompt as input, we generate more natural language utterances for the description. Additionally, outliers are detected and removed. It is followed by the diversity selection to ensure a diverse set of generated utterances. The filtered set of utterances is used for training the intent classifier. Below are the details for each step,

Step 1 (LLM-based utterance generation): LLMs have shown promising results (Zhao et al. 2023) on various language generation tasks. Using a good prompt with few-shot examples, we can use LLMs for text generation without any fine tuning. For utterance generation, we created a text-generation instruction prompt with few-shot examples. As shown in Figure 6A, prompt comprises of *Input*, *Output* fields and we assign agent description as *Input* and the corresponding human provided utterance for the description as *Output*. A total of 12 *Input*, *Output* pairs are provided as few-shot instruction examples by human experts with four agent descriptions each description with three different utterance variations. Along with few-shot examples, the prompt contains a *Input* placeholder for the new description and *Output* will be generated by the LLM for the new description.

In this step, we obtain the agent description and add it in the prompt *Input* placeholder for the new description. We then use this prompt for inference on LLM to generate the *Output*. We perform inference on LLM several times

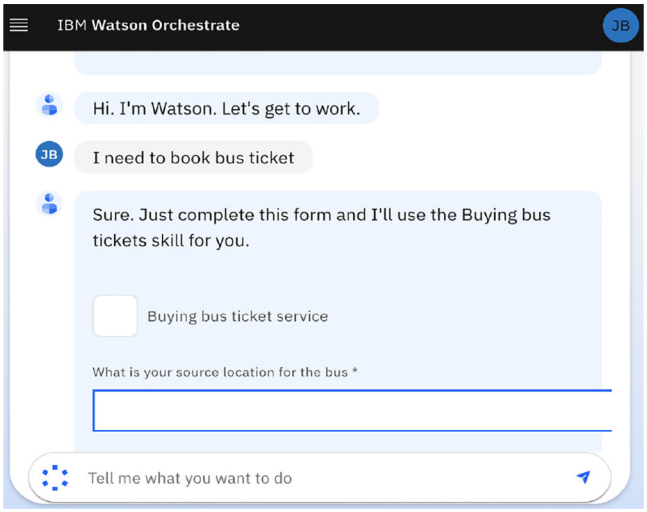


FIGURE 4 System user persona interface.

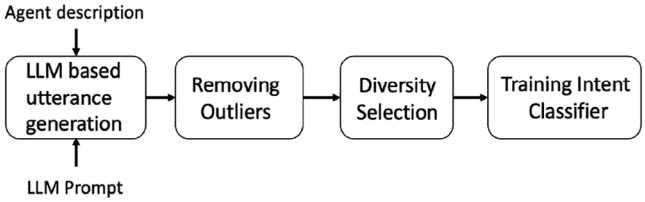


FIGURE 5 Utterance generation.

```
Generate an Output utterance that may be used to
invoke an API using Input sentence
Input: Get the balance of an account
Output: I would like to check the balance of my
savings account
...
...
...
Input: {new_description_placeholder}
Output:
```

(A) Utterance Generation

```
Generate a Question to request following API
Attribute and Description
Description: Buying bus tickets,
Attribute: Source location of the journey,
Question: What is the origin of the bus journey ?
...
...
...
Description:{new_description_placeholder}
Attribute:{new_attribute_placeholder}
Question:
```

(B) Question Generation

FIGURE 6 LLM prompts.

using the same prompt to generate multiple utterances. To generate utterances with variations we used LLM inference parameters such as beam-search with sampling for *decoding strategy*, *temperature* is set to a high value (say, 0.9), *maximum new tokens* is set to 20. And for every agent description, we generate thirty utterances via multiple inference requests to the LLM.

Step 2 (Removing outliers): Utterances generated using the LLM can be noisy and not relevant. To obtain quality utterances by removing outliers from generated utterances, we consider the agent description as a ground truth (GT) seed and use semantic similarity to identify the outliers as below:

(i) Obtain sentence embedding[4] features for agent description, generated utterances using model weights `all-MiniLM-L12-v2`
(ii) For each generated utterance compute the cosine similarity between agent description and utterance
(iii) Remove the utterances that are not semantically similar to the agent description based on the computed cosine similarity score by using the outlier threshold value (say, 0.5). Only utterances with a similarity score above the threshold are selected.

Step 3 (Diversity selection): After removing outliers from the generated utterances, we select a lexically diverse set of utterances as below:

(i) Obtain a dictionary without English stop words using n-gram vocabulary from the utterances and initialize an empty set for the diverse utterances.
(ii) For each utterance, compute the n-gram overlap score by counting the presence of vocabulary of the present utterance in the rest of the generated utterances. And select the utterance that has a high n-gram overlap score, and add it to the set of diverse utterances.
(iii) Iterate over the remaining set of generated utterances, in each iteration compute the n-gram similarity score (Buscaldi et al. 2013) between the current utterance and set of diverse utterances and based on the computed scores, the utterance with least n-gram similarity is added to the set of diverse utterances. Also, during each iteration, we check for the stopping criteria, that is, if the size of the diverse utterances is less than the configured limit for the diverse utterances. Upon finishing the iterations, a set of diverse utterances is obtained.

Step 4 (Training intent classifier): After obtaining diverse utterances from the agent description, we add the utterances as conversational artifacts to the agent

```
{
"openapi": "3.0.1",
  "info": { "description": "Buying bus tickets"}
  "servers": [{"url": "https://prod-server-ticket-
reservation.com"}],
  "paths": {
    "/buyBusTickets": {
        "post": {
            "description": "Buying bus ticket
                        request",
            "properties": {
                "from_location": {"title":"Source
location for the journey"},
                "to_location": {"title":
"Destination location for the journey"},
                "date": {"title": "Date of travel
in DD/MM/YYYY format"}},
            "required": ["from_location",
                       "to_location", "date"]}}}
}
```

FIGURE 7 Example buying bus tickets API.

specification and update the agents in the database. Finally, we train the intent classifier using agent id as intent class and description, diverse utterances as training samples.

In our system, whenever the "builder" persona adds a new API to create an agent, as part of the *build* phase utterance generation is triggered automatically and the intent classifier is trained with augmented training data for the newly created agent. However, the trained intent classifier is used during the *use* phase by the orchestrator for correct agent invocation. Also, the parameters used in different steps of utterance generation such as choice of LLM, LLM inference parameters, number of generated sentences, outlier threshold, n-gram value, and diverse utterance limit can be configured by the "builder."

QUESTION GENERATION

To make an API service call, it is often necessary to provide certain mandatory attributes as inputs to execute the service. Figure 7 shows an example *Buying bus tickets* API where the service operation*/buyBusTickets* requires three fields *from_location, to_location*, and *date* as inputs. While requesting the user to provide inputs for required attributes, asking natural language questions based on the attributes will provide an enhanced conversational experience to the user. Examples of such questions can be, *from_location*: What is the origin of the bus journey?, *to_location*: What is the destination of the bus journey? As described in the previous section, LLMs language generation capabilities can be harnessed for the question generation task. The steps to generate questions using LLMs with API meta-data as input are as given below:

Step 1 (Required attribute collection) : Given an API operation, we extract the list of required attribute names, attributes descriptions from the input schema of the API operation along with the API operation description.

Step 2 (Designing prompt with few-shot examples) : For the question generation task, we design an instruction prompt with few-shot examples. As shown in Figure 6B, prompt comprises of *Description*, *Attribute*, *Question* fields and we assign API operation description as *Description*, attribute description (or attribute name if description is not available) as *Attribute* and the corresponding human-provided question for the attribute as *Question*. Our prompt contains 12 few-shot examples annotated by humans similar to the example shown in Figure 6B. Along with few-shot examples, the prompt contains a *Description* placeholder for the new API operation description, *Attribute* placeholder for the new attribute description and *Question* will be generated by the LLM for the new attribute.

Step 3 (Question generation): Using the list of attributes, prompt from previous steps. For every attribute, we add the attribute description, API operation description in the prompt *Description*, *Attribute* placeholder. And use this prompt for inference on LLM to generate the *Question*. We perform inference with sampling on LLM several times using the same prompt to generate multiple questions. For every attribute, we capture up to three generated questions via multiple inference requests to the LLM.

During the *build* phase, for each agent (API operation), questions are generated for all attributes. Following this, we add the attribute questions as the conversational artifacts to the agent specification and update the agents in the database. The parameters used in different steps of question generation, such as choice of LLM, LLM inference parameters, and number of questions, can be configured by the "builder."

NEXT BEST AGENT RECOMMENDATION

In the *build* phase of an agent (which is referred to as the *target_agent*), we compute the NBAs that are probable candidates to be executed after the *target_agent*. In our system, we recommend the top-*k* NBAs for the *target_agent* to provide a better conversational experience to the user. NBA component of the system at first, extracts data such as description, input, and output signatures from all available agents in the agent database. Using the extracted data, we create a semantic search index for the agents. The purpose of creating an index for the agents is to enable relevance search on all available agents according to the query,

that is, *target_agent* description. For enabling semantic search, we use ChromaDB,[5] which is a vector database that uses `all-MiniLM-L12-v2` model weights to create a search index using the sentence embedding of the agent description as features.

Below we present the algorithm steps to compute NBAs using semantic search index and input–output signature similarity.

Step 1 (Semantic pruning): ChromaDB-based semantic search index supports querying to get relevant APIs from all available agents (S) and we use the cosine similarity to find the most similar top-k agents from S to the *target_agent*.

Step 2 (Input–output match ranking): When agents are sequenced, the data flows from one agent to the next one. Therefore, in this step, we take the top-k agents obtained from Step 1 and rank them based on whether the output of the *target_agent* matches with the input of the candidate NBA agent or not. We provide the final ranking of top-k agents based on the computed input–output matching score. *Calculating Input–output matching score*: We obtain the sentence embeddings of the concatenated attribute descriptions in both the output signature of *target_agent* and the input signature of each candidate NBA agent. We then calculate cosine similarity and use it as an input–output matching score in Step 2.

After obtaining NBAs for the *target_agent*, we add them as conversational artifacts to the agent specification and update the agent specification in the database. The parameters used in different steps of NBA, like the choice of embedding model and the value of top-k can be configured by the "builder" persona.

AUTOMATED TESTING OF AGENT

In the preceding sections, we have demonstrated methods for generating conversational artifacts, such as training utterances for intent classification, and questions for slot-filling, using large language models (LLMs) during the build phase. These artifacts are essential for training conversational agents to handle specific tasks. Once the conversational agent is deployed, it operates without using an LLM during runtime. This ensures that the agent functions relying solely on its pregenerated artifacts to interact with users. However, this brings forth a critical challenge: determining if the generated conversations artifacts would lead to satisfactory conversational experience with an user. To address this challenge, we propose a novel approach that employs LLMs as proxies to simulate human interaction and evaluate the agent's performance. Figure 8 shows the automated testing of an agent, where LLM plays the role of User to converse with the Orchestrator

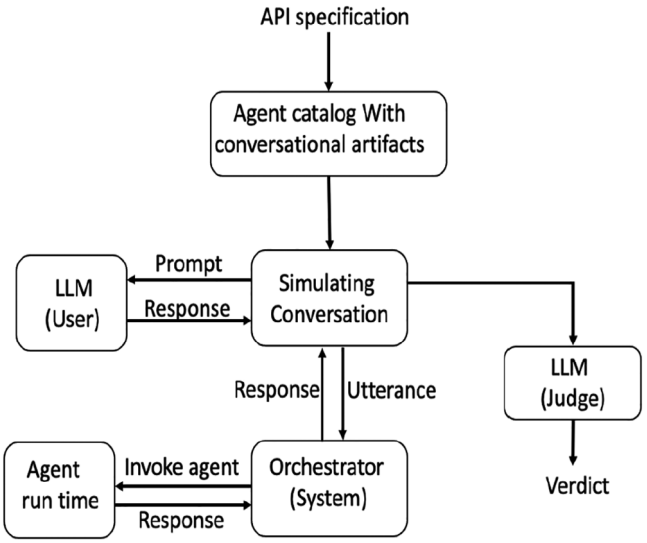


FIGURE 8 Automated agent testing.

```
You must have a conversation with a system and ask it to
perform the following task: Send your location to a
contact
The task requires the following inputs location,
contact_name for which you may provide the relevant
input value when requested. You must provide request to
the system or respond to it's questions after 'User'

System: How may I help you?
User: I want to send my location to a contact
System:  What is the preferred location that you would
like to share for contact purposes?
User: 123 Main St
System:  What is the name of the contact person that
applications should be sent to?
User: John Doe
System: Thank you - ShareLocation is executed
successfully
User: Thank you
```

FIGURE 9 Simulated conversation between system and user.

(System) and the simulated conversation is then evaluated by the LLM acting as a *Judge*. This process involves the following two key steps: Simulation of conversation: We use a prompt to engage an LLM in a conversation with the agent, mimicking a real user's interaction (as a proxy to the user). To begin with, an LLM is prompted to test out a given skill (with a given description). The LLM initiates the conversation by providing an utterance that would necessitate invoking the agent. Further, the LLM provides necessary details through responses to the questions the agent asks. Once the agent completes the conversation, a comprehensive conversational transcript is generated. Figure 9 illustrates a sample LLM prompt followed by the conversation between the proxy user (LLM)

```
Here is a conversation between a user and the system.
The User is trying to perform the following task: Send
your location to a contact which requires only the
following inputs:
1.location,
2.contact_name

System: How may I help you?
User: I want to send my location to a contact
System:  What is the preferred location that you would
like to share for contact purposes?
User: 123 Main St
System:  What is the name of the contact person that
applications should be sent to?
User: John Doe
System: Thank you - ShareLocation is executed
successfully
User: Thank you

Your task is to judge if the System has performed the
task correctly. The system must ask for all the inputs
mentioned above in any order. The system must not ask
for any other information that is not required. Please
explain if the system has performed the task correctly
followed with either 'Verdict: Correct' or 'Verdict:
Wrong
```

FIGURE 10 Judging the conversation.

and the system. Evaluation by LLM judge: The generated transcript is subsequently evaluated by another LLM (Zheng et al. 2023), which acts as an impartial judge. This LLM reviews the conversation and delivers a verdict on whether the agent successfully handled the interaction. The LLM as a judge primarily determines if the correct agent was invoked and if all the necessary attributes we asked for and obtained. If the agent fails the test, the LLM judge provides a detailed explanation of the shortcomings and specific areas where the agent's performance was deficient. Figure 10 shows a sample prompt for an LLM to judge if the agents are able to have a satisfactory conversation with the proxy user and complete the task.

This approach allows for rapid and scalable testing of conversational agents without the need for human testers, thereby speeding up the building process. Further, the detailed explanations provided by the LLM judge in case of failure offer valuable insights for builder, guiding them in making targeted improvements to the agent.

By leveraging LLMs for both simulating user interactions and evaluating agent performance, we effectively assess and enhance the conversational quality of agents during the build phase. This method improves the efficiency of testing processes and ensures that conversational agents deliver satisfactory user experiences when deployed.

VALIDATION

In this section, we present the validation results on the different components of the system. The datasets used for validation are described as follows: **SGD**: Schema Guided Dialog dataset (Rastogi et al. 2020) consists of conversations between a virtual assistant and a user from several domains including Travel, Events, Payment, Media, Restaurants, Weather, and so forth. It includes annotations for natural language understanding, dialog state tracking, API service schema, and user utterances for API invocation. **HR**: HR dataset is an internal dataset, created from the APIs used in various HR operations and the deployed usage data of these APIs. It includes human annotations for the user utterances to invoke API, natural language understanding, and NBA. **ABCD**: Action-based Conversational dataset (Chen et al. 2021) consists of human-to-human dialogs related to user intents along with sequences of actions to achieve a task. It includes annotations for user utterances to invoke an action, natural language understanding, and NBA.

Validation of utterance generation

The Utterance Generation component is used in *build* phase for augmenting training data, training an intent classifier (which is part of the *understand* skill used by the orchestrator). To validate the performance of the Utterance Generation component, we evaluate the intent classifier and shows the value of utterance generation for improving intent classification.

For intent classifier evaluation, we considered two different intent classification algorithms: (i) Watson Assistant (WA)[6] and (ii) XGBoost.[7] The intent classifiers are trained and tested on two datasets (i) HR and (ii) SGD with utterance generation using different LLMs.

Datasets: HR dataset consists of 22 APIs and for the intent classification task, we processed the API specifications to obtain the required data, which consists of 22 API descriptions and 482 human annotated utterances to invoke the API. Similarly, for the SGD (Rastogi et al. 2020) dataset, we processed the dialogs from the test-set split and extracted the first turn of each dialog to obtain user utterance along with associated API. Processed SGD test-set consists of 26 API descriptions and 4201 user utterances. In both datasets, API descriptions are the agent description and user utterances are used as test samples for intent classification.

LLMs: We evaluate several open source LLMs such as FLAN-T5-XXL (Chung et al. 2022), FLAN-UL2 (Tay et al. 2022), Llama2-13b (Touvron et al. 2023), Llama-3-70B-Instruct (AI@Meta Llama 3 Model Card 2023), and

TABLE 1 Intent classification results.

		HR				SGD			
Classifier	**LLM**	***Accuracy***	***Incorrect***	***Fallback***	***F1***	***Accuracy***	***Incorrect***	***Fallback***	***F1***
WA	Baseline	0.53	0.28	0.19	0.53	0.35	0.55	0.10	0.49
WA	FLAN-T5-XXL	0.66	0.27	0.07	0.67	0.71	0.15	0.14	0.72
WA	FLAN-UL2	0.59	0.31	0.10	0.61	0.68	0.17	0.15	0.67
WA	Llama2-13b	0.65	0.29	0.06	0.65	0.68	0.18	0.14	0.71
WA	Llama-3-70B-Instruct	0.67	0.26	0.07	0.63	0.72	0.18	0.10	0.69
WA	Mixtral-8x7B-Instruct-v0.1	0.73	0.20	0.07	0.71	0.75	0.15	0.10	0.73
XGBoost	Baseline	0.04	0.86	0.10	0.02	0.02	0.76	0.22	0.02
XGBoost	FLAN-T5-XXL	0.52	0.40	0.08	0.50	0.51	0.46	0.03	0.53
XGBoost	FLAN-UL2	0.49	0.47	0.04	0.48	0.43	0.39	0.18	0.47
XGBoost	Llama2-13b	0.46	0.45	0.09	0.45	0.51	0.35	0.14	0.53
XGBoost	Llama-3-70B-Instruct	0.55	0.41	0.04	0.53	0.57	0.35	0.08	0.59
XGBoost	Mixtral-8x7B-Instruct-v0.1	0.58	0.36	0.06	0.56	0.61	0.30	0.09	0.60

Mixtral-8x7B-Instruct-v0.1 (Jiang et al. 2024) for training data augmentation

Metrics: Intent classifier performance is evaluated using the following metrics: (i) *Fallback*: the ratio of the number of Fallback predictions and the total number of predictions, (ii)*Accuracy*: the ratio of the number of correct predictions and the total number of predictions, (iii)*F1-score*: the harmonic mean of Precision and Recall; and the (iv)*In-correct*: the ratio of number of predictions that are neither *correct* nor *Fallback* and the total number of predictions. It represents the actual misclassification rate.

In our evaluation, each of the intent classifier algorithms is trained in four variations, that is, the Baseline model uses only descriptions as training samples and the other three variations use different LLMs for generating utterances to augment the training data. The intent classifiers are trained using WA, which is a proprietary algorithm and provides SDK (software development kit) for train and predict functionalities where whereas XGBoost is an open-source algorithm trained using tf-idf (Ramos et al. 2003) features with five-fold cross-validation to find optimal hyper-parameters. We use different variations of trained intent classifiers to predict the agent for the test user utterances, if the model prediction probability score is lower than the confidence threshold it will predict the intent class as "Fallback," that is, Orchestrator will not select an agent. It is important to assign a valid threshold for "Fallback" and the optimal value for the threshold is determined during the classifier training using five-fold cross-validation.

Validation results are shown in Table 1, for both the classifier algorithms, the LLM utterance generation results are better than the Baseline in terms of *Accuracy*, *F1-score*, *In-correct* metrics.

TABLE 2 Accuracy of question generation as compared to ground-truth questions.

LLM	**QG accuracy (SGD)**
FLAN-T5-XL	0.734
FLAN-T5-XXL	0.719
FLAN-UL2	0.742
Llama-2-13b	0.609
Llama-2-13b-Chat	0.797
Llama-3-70B-Instruct	0.828
Mixtral-8x7B-Instruct-v0.1	0.883

Validation of question generation

The Question Generation component is evaluated on the SGD (Rastogi et al. 2020) dataset. This dataset contains dialogs with API schema attributes associated with user questions in the conversation between the virtual assistant and the user. We processed the test-set split of the dataset to obtain the API service description, schema attribute description, and associated questions asked by the virtual assistant. Through human annotations, we hand-picked only the samples with high quality. In this curated test data, there are 128 sample records with *Description*, *Attribute*, and *Question*. We evaluated several LLMs as shown in Table 2 to generate questions on the test data.

As described in section *Question Generation*, for each sample in the test set, we obtain the few-shot examples prompt with API description and attribute description populated in the *Description* and *Attribute* placeholders. Using this prompt, we perform inference on the LLM to generate the *Question* for the given attribute. To evaluate the quality of the generated questions, they need to be compared with the GT questions in the curated dataset.

Of course, the generated questions do not have to exactly match, but we like them to be semantically similar. The ideal way to make this comparison would be for human experts to annotate the results, indicating whether each of the questions is valid/usable or not. However, due to the large number of samples, annotation of all samples is cumbersome and not viable.

Our approach is to estimate the semantic similarity between the generated question GQ_i and the GT question Q_i for each sample i. Using sentence embedding[8] of the questions from the `all-MiniLM-L12-v2` model weights, and the cosine similarity is computed (denoted as $S(GQ_i, Q_i)$). While the similarity between the generated and GT questions can be computed, it still does not indicate if the questions were usable.

To alleviate this issue, a subset of the generated questions (around 15%) were annotated by human experts with each sample, indicating whether the generated question was usable or not. This annotation is used to calibrate a model-specific threshold λ on the similarity score. Any generated question for which $S(GQ_i, Q_i) > \lambda$ is deemed to be good. The optimal threshold λ is obtained based on k-fold cross-validation on the annotated subset to best match the human annotation.

Once the similarity scores are calculated for all 128 test samples with that of the GT questions, we compute the fraction of cases for which the generated questions have a similarity score greater than the threshold λ. This is referred to as the accuracy of question generation (QG-accuracy). This also reflects the effectiveness of the underlying LLM for the task of question generation.

Table 2 shows the QG-accuracy of various LLMs. It can be observed that the questions generated through Mixtral-8x7B-Instruct-v0.1 (Jiang et al. 2024) are very similar to the human annotated GT questions.

Validation of next best agents

Our proposed NBA recommendation algorithm is validated using two datasets: HR and ABCD. In the HR dataset, we have 22 APIs/agents with the description and NBA annotations. For the ABCD dataset, we processed the raw data to obtain 30 different actions and their NBAs.

Metrics: To evaluate the performance of NBA, we define the following two evaluation metrics: i)*NBA accuracy*: the fraction of total number of NBAs predicted correctly and the total number of NBAs present in the GT across all agents, (ii)*Agent Recall*: the fraction of number of agents for which at least one NBA was predicted correctly in top-k out of the total number of recommended agents.

We evaluate the performance of our approach using Elastic Search based on the baseline and it supports Keyword based search. In the Elastic Search baseline, we first obtain features using BM25 (Robertson and Zaragoza 2009) method for all the agent descriptions, and the search index is created using the BM25 features. This Keyword-based search index supports querying the relevant top-k APIs based on API description and the ranking of APIs is computed using the cosine distance between tf-idf (Ramos et al. 2003) features of *target_agent* and all agents in the index. We use $k = 5$ in our experiments and the results obtained from our proposed approach are shown in Table 3. As shown, our approach outperforms the Elastic Search baseline on the ABCD dataset and performs comparably on the HR dataset. This is because the API descriptions in the HR dataset contain high keyword overlap that Elastic Search is able to handle in a better way. However, our approach would scale better for more natural agent descriptions.

TABLE 3 Next best agent results.

Method	NBA Accuracy		Agent Recall	
	ABCD	*HR*	*ABCD*	*HR*
Our approach	0.26	0.39	0.50	0.61
Elastic search	0.10	0.40	0.27	0.58

Validation of system (end-to-end) using automated agent testing

The previous subsections detailed the independent validation of each type of conversational artifact. Here, we present the outcomes of our automated agent testing component, which validates the combined artifacts and assesses the overall conversational experience.

For this experiment, we utilized the full SGD dataset with train, test, and validation sets combined. In the full data set, there are 46 services, each with its own descriptions and input parameter slots. We generated conversational artifacts for these services and instantiated agents within our system. Due to the absence of actual runtime for the SGD services, we mocked the service APIs using FastAPI[9] framework to return dummy response data.

To simulate the conversation of a user with the agents, we employed LLM Mixtral-8x7B-Instruct-v0.1 (Jiang et al. 2024) to initiate a conversation invoking each service. The resulting conversation transcripts were then evaluated by a judge, which is also a Mixtral-8x7B-Instruct-v0.1 LLM with the task of a judge. This model assessed whether the agents correctly identified the intent and gathered all necessary data for service invocation. In instances of failure, the judge LLM provided explanations to the *builder* for further analysis and improvement.

In the experiment, there are 46 agents corresponding to 46 services in the SGD data. Out of the 46 agents, for 45 agents, the judge LLM's verdict is returned a "Correct" and

the service APIs are invoked successfully with all necessary inputs, that is, approximately *98%* of services in the SGD data set. However, in one case, the judge LLM's verdict is "Wrong" because the orchestrator (System) could not correctly identify the agent's intent and no service API was invoked.

DEPLOYMENTS DETAILS

Our system is deployed and used as part of the Watson Orchestrate[10] product with general availability to the customers since December 2022, also the product is honored with CES Innovation Award 2022.[11] To date, the product has been deployed in over 2500 tenants and used by more than 11,000 active users. Over 600 APIs have been wrapped up as agents by *builders*. Further, these agents have been invoked over 220,000 times by *users*. In this paper, we evaluated the system of Watson Orchestrate focused on HR process use-case with APIs related to candidate sourcing, career mobility, HRs, onboarding, procurement, recruiting, and talent acquisition. The product offers out-of-the-box support for several business-critical APIs and RPAs services with a conversational interface enabled. The services include Asana, Dropbox, Oracle HC, Salesforce, SAP SuccessFactors, IBM RPA, ThisWay Global, Jira, Slack, Gmail, and so forth.

For the system development, we used Python for the back-end components deployed on the Openshift cloud platform[12] as a multitenant service. Openshift platform provisions different hardware requirements for various components. For deploying the LLMs, we leverage the NVIDIA A100 GPUs and for the other components, CPUs are sufficient. We use asynchronous inference APIs using FastAPI[13] framework for different LLMs such that all the other components of the system can make REST API calls to the LLM inference service. The other components of the system are packaged as different docker images with each handling different functionalities such as Utterance Generation, Question Generation, NBA, Orchestrator, and Database. In the deployment system, components can be configured and managed independently from each other. We use an automated DevOps pipeline for system deployment and maintenance. Maintenance includes deploying new releases for the system components with enhancements and bug fixes from product usage.

CONCLUSIONS AND FUTURE WORK

Generating conversational artifacts such as sample utterances for data augmentation, question generation for slot-filling and next action predictions can simplify the process of creating chatbots and improving the user experience, particularly for those without the necessary AI expertise. Our methods demonstrated their effectiveness by producing significant accuracy improvements across various datasets. These methods were also successfully integrated into Watson Orchestrate Product. Nevertheless, there are limitations to this approach that we intend to tackle in our future endeavors. Specifically, we would like to utilize human-in-the-loop validation to enhance our methods and capitalize on end-user feedback to further enrich the dataset and facilitate model retraining. The performance of the NBA can further be improved by leveraging the process knowledge and the usage data collected from the proposed system.

CONFLICT OF INTEREST STATEMENT

The authors declare that there is no conflict.

ORCID

Jayachandu Bandlamudi https://orcid.org/0009-0002-3032-042X

ENDNOTES

[1] Digital Assistant, Conversational Assistant, and Chatbot are used interchangeably.
[2] https://spec.openapis.org/oas/latest.html
[3] APIs refer to the OpenAPI specifications from here on.
[4] https://www.sbert.net/
[5] https://github.com/chroma-core/chroma
[6] https://www.ibm.com/products/watson-assistant
[7] https://xgboost.readthedocs.io/en/stable/python/
[8] https://www.sbert.net/
[9] https://fastapi.tiangolo.com/
[10] https://www.ibm.com/products/watson-orchestrate
[11] https://www.ces.tech/innovation-awards/honorees/2022/honorees/w/watson-orchestrate.aspx
[12] https://www.redhat.com/en/technologies/cloud-computing/openshift
[13] https://fastapi.tiangolo.com/

REFERENCES

AI@Meta Llama 3 Model Card. 2024. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.

Bang, Y., S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Wilie, H. Lovenia, Z. Ji, T. Yu, W. Chung, Q. V. Do, Y. Xu, and P. Fung 2023. "A Multitask, Multilingual." Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity (Version 4). arXiv. https://doi.org/10.48550/ARXIV.2302.04023

Bocklisch, T., J. Faulkner, N. Pawlowski, and A. Nichol 2017. Rasa: Open Source Language Understanding and Dialogue Management (Version2). arXiv. https://doi.org/10.48550/ARXIV.1712.05181

Buscaldi, D., J. Le Roux, J. Garcıa Flores, and A. Popescu. 2013. "LIPN-CORE: Semantic Text Similarity Using n-Grams, WordNet, Syntactic Analysis, ESA and Information Retrieval based Features." In *Second Joint Conference on Lexical and Computational*

*Semantics (*SEM), Volume 1: Proceedings of the Main Conference and the Shared Task: Semantic Textual Similarity*, 162–68. https://aclanthology.org/S13-1023

Chen, D., H. Chen, Y. Yang, A. Lin, and Z. Yu. 2021. "Action-Based Conversations Dataset: A Corpus for Building More In-Depth Task-Oriented Dialogue Systems." In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021*, 3002–17. https://www.aclweb.org/anthology/2021.naacl-main.239.

Chung, H. W., L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, … J. Wei 2022. Scaling Instruction-Finetuned Language Models (Version 5). arXiv. https://doi.org/10.48550/ARXIV.2210.11416

Devarajan, Y. 2018. "A Study of Robotic Process Automation Use Cases Today for Tomorrow's Business." *International Journal Of Computer Techniques* 5: 12–18.

Jiang, Albert Q., Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, et al. 2024. "Mixtral of Experts." https://arxiv.org/abs/2401.04088.

Laurent, P., T. Chollet, and E. Herzberg. 2015. "Intelligent Automation Entering the Business World." Deloitte. https://www2.deloitte.com/content/dam/Deloitte/mt/Documents/rpa/dt-Automation-with-intelligence.pdf (accessed March 5, 2018).

Patil, S. G., T. Zhang, X. Wang, and J. E. Gonzalez 2023. Gorilla: Large Language Model Connected with Massive APIs (Version 1). arXiv. https://doi.org/10.48550/ARXIV.2305.15334

Qi, H., L. Pan, A. Sood, A. Shah, L. Kunc, M. Yu, and S. Potdar. 2021. "Benchmarking Commercial Intent Detection Services with Practice-Driven Evaluations." In *NAACL*.

Ramos, J. 2003. "Using TF-IDF to Determine Word Relevance in Document Queries." In *Proceedings of the First Instructional Conference on Machine Learning*, vol. 242, 29–48.

Rastogi, A., X. Zang, S. Sunkara, R. Gupta, and P. Khaitan. 2020. "Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset." In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 8689–96.

Rizk, Y., V. Isahagian, S. Boag, Y. Khazaeni, M. Unuvar, V. Muthusamy, and R. Khalaf. 2020. "A Conversational Digital Assistant for Intelligent Process Automation." In *BPM 2020 Blockchain And RPA Forum*, vol. 393, 85–100.

Robertson, S., and H. Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval* 3: 333–89.

Sabharwal, N., S. Barua, N. Anand, and P. Aggarwal. 2020. "Building Your First Bot Using Watson Assistant." In *Developing Cognitive Bots Using the IBM Watson Engine: Practical, Hands-on Guide to Developing Complex Cognitive Bots Using the IBM Watson Platform*, 47–102. Berkeley: Apress. https://doi.org/10.1007/978-1-4842-5555-1_4

Syed, R., S. Suriadi, M. Adams, W. Bandara, S. J. J. Leemans, C. Ouyang, A. H. M. ter Hofstede, I. van de Weerd, M. T. Wynn, and H. A. Reijers. 2020. "Robotic Process Automation: Contemporary Themes and Challenges." *Computers in Industry* 115: 103162.

Tay, Y., M. Dehghani, V. Q. Tran, X. Garcia, J. Wei, X. Wang, H. W. Chung, S. Shakeri, D. Bahri, T. Schuster, H. S. Zheng, D. Zhou, N. Houlsby, and D. Metzler 2022. UL2: Unifying Language Learning Paradigms (Version 3). arXiv. https://doi.org/10.48550/ARXIV.2205.05131

Touvron, H., L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, … T. Scialom 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models (Version 2). arXiv. https://doi.org/10.48550/ARXIV.2307.09288

Vaziri, M., L. Mandel, A. Shinnar, J. Siméon, and M. Hirzel. 2017. "Generating Chat Bots from Web API Specifications." In *Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 44–57.

Yaghoub-Zadeh-Fard, M., B. Benatallah, and S. Zamanirad. 2020. "Automatic Canonical Utterance Generation for Task-Oriented Bots from API Specifications." In *EDBT*.

Yang Y., M. C. S. UY, and A. Huang 2020. "FinBERT: A Pretrained Language Model for Financial Communications (Version 2)." arXiv. https://doi.org/10.48550/ARXIV.2006.08097

Zhao, W. X., K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, … J.-R. Wen 2023. "A Survey of Large Language Models (Version 14)." arXiv. https://doi.org/10.48550/ARXIV.2303.18223

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, et al. 2023. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." https://arxiv.org/abs/2306.05685.

How to cite this article: Bandlamudi, J., K. Mukherjee, P. Agarwal, R. Chaudhuri, R. Pimplikar, S. Dechu, A. Straley, A. Ponniah, and R. Sindhgatta. 2024. "Framework to enable and test conversational assistant for APIs and RPAs." *AI Magazine* 45: 443–56. https://doi.org/10.1002/aaai.12198

AUTHOR BIOGRAPHIES

Jayachandu Bandlamudi is an Advisory Research Engineer at IBM Research, India. He received his Master's degree from the University of Illinois at Urbana-Champaign, he has over 9 years of experience in Research and Development of scalable AI systems. Currently, he is working in the area of AI for Business Automation with research interests in machine learning, deep learning, and natural language processing.

Kushal Mukherjee is a Senior Research Scientist at IBM Research, India. He received his PhD from Pennsylvania State University and has over 13 years of industry experience working with AI and its applications. He is proficient in the areas of large language models

(LLMs), collaborative intelligence, AI for business processes, workforce analytics, and time-series forecasting.

Prerna Agarwal works as an Advisory Research Engineer at IBM Research, India. She has completed her Master's from IIIT Delhi, India. She has over 9 years of experience in Industry and Academia in Artificial Intelligence. She is currently working in the area of business automation and Artificial Intelligence. Her research interests in Machine Learning, NLP, and Question Answering systems.

Ritwik Chaudhuri is a Staff Research Scientist at IBM Research, India, with nearly a decade of industry experience. His expertise spans traditional machine learning, deep learning, and time series forecasting. Additionally, he is proficient in areas of large language models (LLMs).

Rakesh Pimplikar is a Senior Solution Architect at IBM Software, with 14 years of industry experience developing AI solutions. He is proficient in the areas of generative AI, natural language processing, and business process automation.

Sampath Dechu is a Principal Product Manager for Digital Labor product suite at IBM Software. He received his Master's degree from Iowa State University in 2003. Before joining IBM, he worked at multiple early stage startups building optimization software for high performance microprocessor applications. At IBM, prior to his current role as product manager, Sampath was part of IBM Research, leading team of research scientists and software engineers to build AI applications for business process automation and optimization. He is proficient in the areas of natural language processing, generative AI, business process monitoring, and optimization.

Alex Straley is a Product Manager for IBM Software. He received his MBA and Masters in Information Systems from Boston University. Alex has worked in IT, HR, and product management. He has built products for healthcare, career and salary management, mergers and acquisitions, and more. He now focuses on AI-driven product within the digital labor space.

Anbumunee Ponniah is an Open Group certified distinguished IT Specialist and a Chief Architect with IBM Software responsible for creating AI-driven automation products. He has 28 years of IT experience architecting products and solutions driving digital transformation and AI strategies for customers across retail, telecom, insurance, and government sectors. His expertise is in industry applications of AI, integrating heterogeneous systems, and automation of business processes. He completed his doctorate in NLP-based business process analysis from BITS Pilani, Goa, in 2024.

Renuka Sindhgatta is a Senior Technical Staff Member and Research Manager at IBM Research, India. She received her PhD from the University of Wollongong. She has research interests in the area of process analytics, interpretable predictive analytics, and focuses on supporting automation via a hybrid workforce of intelligent agents and humans to accelerate business processes automation.

Copyright of AI Magazine is the property of AI Magazine and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.